

Module 28: Advanced Inheritance

Introduction to C++ Programming — Part 3: Object-Oriented Programming with C++

Module Overview

Having argued in Module 27 for composition over inheritance, this module teaches students to use inheritance well when it is the right tool. It fills in what Module 11 deliberately skipped: the three inheritance modes (and why public is the only one that means is-a), inheriting constructors with using Base::Base, the surprising name-hiding rule and its using-declaration fix, final on methods and classes, object slicing in all its forms, and multiple inheritance - ending with the diamond problem and virtual inheritance, including the rule that the most-derived class constructs the shared base.

Suggested Duration: 90 minutes

Learning Objectives

By the end of this module, students will be able to:

- Explain public, protected and private inheritance, and prefer composition to private inheritance.
- Inherit a base class's constructors with using Base::Base.
- Recognise name hiding and fix it with a using-declaration.
- Use final on virtual methods and on classes, and justify when to do so.
- Identify the three forms of object slicing and prevent them.
- Use multiple inheritance, resolve ambiguities, and solve the diamond problem with virtual inheritance.

Prerequisites

- Part 2 Modules 11-12 (inheritance and polymorphism).
- Module 27 (composition over inheritance) - this module assumes students now reach for inheritance deliberately.

Materials & Instructor Prep

- 28-advanced-inheritance/examples/ 01 to 07; 03_name_hiding.cpp and 07_diamond_virtual_inheritance.cpp are the key demos.
- The exercise deliberately does not compile; have it open so you can demonstrate fixing errors one at a time.

Key Vocabulary

Term	Definition
Public inheritance	Inheritance meaning is-a: the base's public members stay public and the derived object can be used wherever the base is expected.
Private inheritance	Inheritance meaning 'implemented in terms of': the base's members become private and outsiders can't treat the object as a base.
Inheriting constructors	using Base::Base; makes all of the base class's constructors available in the derived class.
Name hiding	Declaring a function in a derived class hides every base function with the same name, whatever their parameters.
final	On a virtual method: no further overriding. On a class: no further deriving.
Diamond problem	A class inheriting from two classes that share a base ends up with two copies of that base.
Virtual inheritance	Declaring a base virtual (class B : virtual public A) so only one shared copy exists; the most-derived class constructs it.

Lesson Outline & Instructor Talking Points

1. Inheritance modes (15 min)

- Run examples/01_inheritance_access_modes.cpp and uncomment the failing lines one at a time to show what each mode hides.
- Show the composed LessonClock next to the privately inheriting one and ask which a new teammate would understand faster.
- Point out the trap: for a class, leaving out the word public means PRIVATE inheritance.

2. Inheriting constructors and name hiding (20 min)

- Run examples/02_inheriting_constructors.cpp; note that new members need default member initializers.
- Before running examples/03_name_hiding.cpp, ask whether h.log(404) will compile. Most say yes. Show the error, explain that name lookup stops at the first scope containing the name, then fix it with using Logger::log.

Instructor note: Name hiding happens before overload resolution and before virtual dispatch. Say this explicitly - students otherwise try to fix it with virtual.

3. final and slicing revisited (15 min)

- Run examples/04_final.cpp; uncomment the overriding emergencyStop to show the error. Discuss safety-critical behaviour as a reason for final.
- Run examples/05_slicing_revisited.cpp and name the three forms of slicing. Ask how Module 26 already prevents them (containers of unique_ptr).

4. Multiple inheritance (15 min)

- Run examples/06_multiple_inheritance.cpp and trace construction order. Show a qualified call, then argue that Smartphone defining its own status() is the better fix.

5. The diamond and virtual inheritance (25 min)

- Draw the diamond. Run examples/07_diamond_virtual_inheritance.cpp: the plain version constructs two Devices with two serial numbers.
- Show the fixed version, stressing the two rules: virtual on BOTH middle classes, and the most-derived class constructs the shared base.
- Close with the guideline: multiple inheritance of interfaces is safe; diamonds of classes with data deserve a second look.

Instructor note: Keep virtual inheritance in proportion. Students need to recognise and correctly build a diamond, but should leave believing that they will rarely need one.

Common Student Misconceptions

Misconception	How to Address It
"class Resistor : Component is the same as : public Component."	For a class, the default is private inheritance: nothing from Component is visible outside and the Resistor can't be used as a Component.
"Overriding one overload in the derived class keeps the others."	Declaring the name hides every base overload with that name. Bring them back with a using-declaration.
"virtual inheritance is the same as virtual functions."	Same keyword, unrelated meaning: virtual inheritance shares one base subobject; virtual functions enable dynamic dispatch.
"Multiple inheritance should always be avoided."	Inheriting several interfaces (no data) is common and safe. The risk is mainly diamonds of classes that hold data.

Exercises & Module Project

Exercise: 28-advanced-inheritance/exercises/exercise1.cpp - a school weather station that does not compile until students fix the inheritance mode, replace forwarding constructors, cure name hiding, add final, and resolve a diamond with virtual inheritance. Project (three tracks): Generic - Person/Teacher/Researcher/TeachingResearcher; EE - Device/PoweredDevice/NetworkedDevice/SmartPlug; Biomedical - MedicalDevice/Wearable/WirelessDevice/SmartWatchMonitor. Each builds a real diamond correctly and uses final, using-declarations and inheriting constructors.

Facilitation guidance: Check the trace line in the top class's constructor appears exactly once per bottom-class object - that proves virtual inheritance is working. Read the composition-check comment carefully: a student who argues convincingly that composition would have been simpler has understood the module.

Discussion & Comprehension Check Questions

1. Why does adding `log(std::string)` to a derived class stop `log(int)` from the base compiling?

Guidance: Name lookup finds 'log' in the derived class and stops looking, so the base's overloads are never considered. using `Base::log`; brings them into the derived scope.

2. In a diamond with virtual inheritance, who constructs the shared base and why?

Guidance: The most-derived class, because the shared base belongs to neither middle class alone; their own calls to its constructor are skipped.

3. When would you mark a method `final`?

Guidance: When further overriding would break an important rule - for example a safety-critical shutdown - or to document that a design decision is finished.

Assessment Ideas

- Compiler-error clinic: show three errors (private-by-default inheritance, hidden overload, final violation) and ask for the cause and fix of each.
- Ask students to draw a diamond from their own track and state, with reasons, whether composition would have been better.

Answer Key Notes

See 28-advanced-inheritance/exercises/solutions/exercise1_solution.cpp. Note the starter exercise intentionally does not compile.

Wrap-Up & Bridge to Next Module

Bridge forward: "Several of our base classes have been marked = 0 since Module 26. Next module covers abstract classes and interfaces properly - plus vtables, `dynamic_cast`, and polymorphic copying."

Instructor Tips

- If time is short, protect name hiding (step 2) and the diamond (step 5); inheritance modes can be summarised from the table.
- Let students fix the exercise's compile errors in order; reading errors is a skill worth practising.